

Red Spotify

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

CS 4308 Senior Project

Spring Semester 2026

Professor Perry

02/08/2026

1.0	Introduction	3
1.1	Overview	3
1.2	Project Goals	3
1.3	Definitions and Acronyms	3
1.4	Assumptions	3
2.0	Design Constraints	4
2.1	Environment	4
2.2	User Characteristics	4
2.3	System	4
3.0	Functional Requirements	4
3.1	Account Setup and Login	4
3.2	Spotify Authorization and Permissions	4
3.3	Search and Browse	5
3.4	Playback Control	5
4.0	Non-Functional Requirements	5
4.1	Security	5
4.2	Capacity	5
4.3	Usability	5
4.4	Other	6
5.0	External Interface Requirements	6
5.1	User Interface Requirements	6
5.2	Hardware Interface Requirements	6
5.3	Software Interface Requirements	6
5.4	Communication Interface Requirements	6

Table of Contents

1.0 Introduction

1.1 Overview

This application is inspired by Spotify and focuses on the improvement of music discovery, personalization, and sociability through curated content and analytics. Red Spotify integrates the Spotify Web API to collect music meta data, playback information, user listening data, playlists, and audio analysis from users. The application is built on React Native and uses the Spotify OAuth authentication. Ultimately, this application will not replace Spotify but allow the user to be able to better discover music and visualize their listening habits through an intuitive and modern interface.

1.2 Project Goals

The goals of this application include:

- Enhancing music discovery through personal recommendations
- Analytics of users' music habits
- Customization on what music data appears on users' profile
- Provide intuitive, modern, and clean UI/UX
- Utilize Spotify Web API

1.3 Definitions and Acronyms

- **SRS** – Software Requirements Specification
- **API** – Application Programming Interface
- **UX** – User Experience
- **UI** – User Interface
- **OAuth (Open Authorization)** – an authorization framework that allows an application to be authorized to access a resource
- **Spotify Web API** – Spotify's public API for accessing music and user data

1.4 Assumptions

- Users have an existing Spotify account
- Access to internet connectivity
- Music data and playback are governed by Spotify API limitations

2.0 Design Constraints

- The application must integrate the Spotify Web API.
- User authentication must be done through Spotify OAuth 2.0.
- The application must comply with Spotify API usage policies and rate limits.
- The application will not stream the audio (this is done through Spotify) but will display listening habits and new music for the user to listen to.

2.1 Environment

- The application is developed using React Native
- Internet access connection is required to communicate to Spotify Web API
- The application will run on
 - iOS devices (14 or higher)
 - Android devices (10 or higher)

2.2 User Characteristics

- Users should be active Spotify listeners
- Users should be familiar with modern smartphone navigation
- Users have varying degrees of technical expertise

2.3 System

- React Native for front end
- Spotify Web API for music data, user data, and playback functionality
- Spotify OAuth for user authentication
- Potentially lightweight backend for data handling

3.0 Functional Requirements

3.1 Account Setup and Login

- Create Account
- Login with Email and Password
- Login with Spotify (OAuth)
- Logout
- Password Recovery
- Session Timeout and Auto Re-Login

3.2 Spotify Authorization and Permissions

- Connect Spotify Account via Spotify OAuth
- Request Required Scopes (Playback, Library, Playlists, Profile)
- Display Authorization Status (Connected or Not Connected)

- Revoke Spotify Access (Disconnect)
- Handle Expired Tokens (Refresh Token Flow)
- Show Error When Permissions Are Denied or Missing

3.3 Search and Browse

- Search Tracks, Artists, Albums, and Playlists
- Filter Search Results by Type
- Display Track Details (Title, Artist, Album, Duration)
- Display Extra Track Information (BPM, Key, etc.)
- Display Artist Details (Top Songs, Albums, Related Artists)
- Display Album Details (Track list, Release Date)
- Browse Categories (Genres, Moods, Charts)
- Display Related Artists to a Specific Artist

3.4 Playback Control

- Play Track on Selected Device
- Pause/Resume Playback
- Skip Next / Previous
- Seek Within Track
- Shuffle Toggle
- Repeat Toggle (Off, Track, Context)
- Display Now Playing Screen (Cover Art, Progress, Controls, Track Number)
- Display Next in Queue
- Allow Queue sorting
- Handle Playback Restrictions

3.5 User Listening/Profile Information

- Display User Listening Information (Minutes, Top Artists, Top Tracks, etc.)
- Allow User Information Dashboard Customization
- Allow Users to Select Top Artists and Albums to Pin to Dashboard

3.6 Playlist and Library Customization

- Allows Users to Edit Playlists and Playlist Info
- Search Tracks, Artists, Albums, and Playlists in Player Library
- View Entire User Track Library
- Allow Users to Sort Library by Track Info

4.0 Non-Functional Requirements

4.1 Security

- Users will be informed on what data is being collected.
- The application will comply with Spotify API usage and data privacy policies.
- User authentication will be handled with Spotify OAuth

- The application will expire inactive sessions.

4.2 Capacity

- The application will not have reduced performance with many users.
- Users will be able to have growth with their listening data without data problems.
- The application should be able to handle multiple Spotify API requests.

4.3 Usability

- The interface should be easy to use and smooth when using the application.
- The application should provide easy to see visuals for habits and trends.
- The users interactions should have little steps to navigate around the app to where the user wants to go.
- The application will follow typical mobile UI/UX designs.

4.4 Other

- The application will be able to run on either iOS or Android.
- The application will be able to handle any Spotify API downtime.
- Error messages will display when the system fails.
- The system will only collect data that is required for it to function.

5.0 External Interface Requirements

5.1 User Interface Requirements

- The application will have a mobile friendly graphical user interface.
- The application will present analytics for the users listening habits.
- There will be minimal user interaction to navigate to music discovery and other analytical features.
- Error and loading messages will be clearly displayed to the user.

5.2 Hardware Interface Requirements

- The application will be able to run on mobile phones / tablets that are with iOS or Android.
- The devices the application will run on should support touch screen inputs.

5.3 Software Interface Requirements

- This application will use the Spotify Web API to get music data, playlists, and listening history.
- The application will use Spotify OAuth for user authentication and authorization.

- The application will interface with iOS and Android devices.
- The application will be made with React Native.

5.4 Communication Interface Requirements

- The application will communicate with external services by using HTTP.
- All data exchanged with the Spotify Web API will use the JSON format.
- The application will need an internet connection to operate properly.
- The system will handle network interruptions well.